

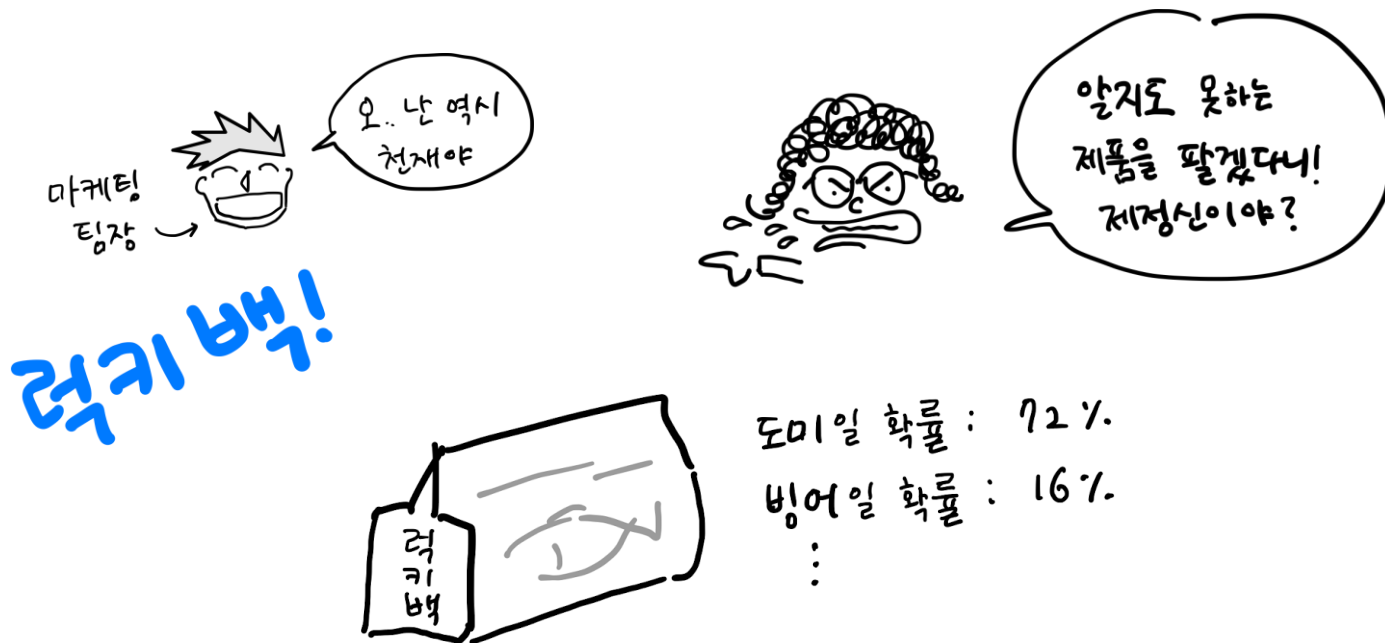
다양한 분류 알고리즘

• 1. 로지스틱 회귀

럭키백



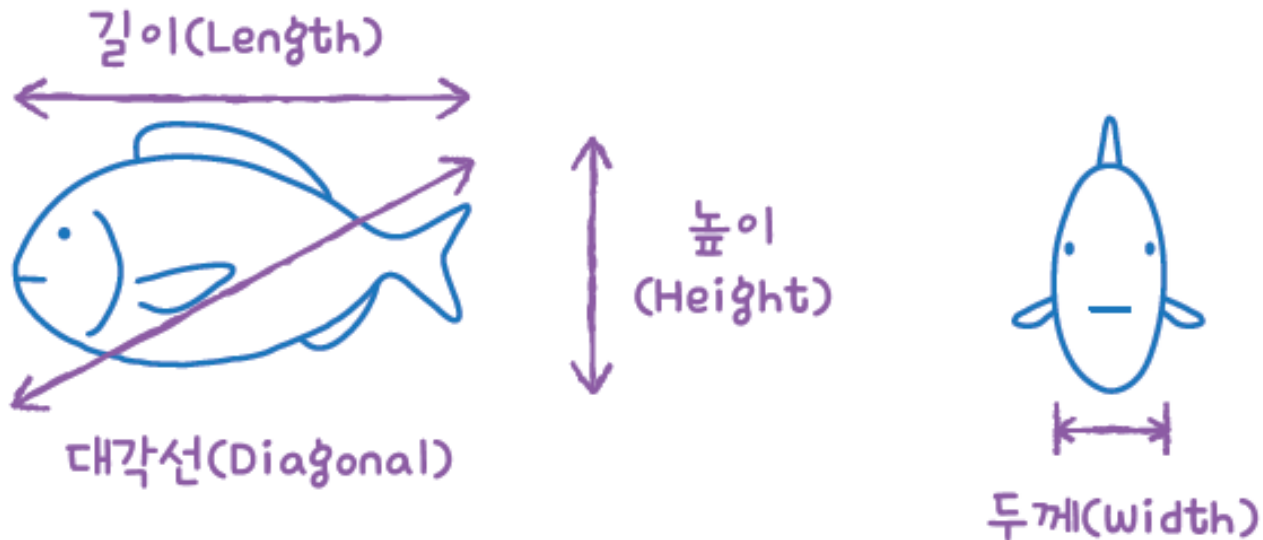
- 마케팅팀은 다가오는 명절 특수에 고객의 이목을 끌 새로운 이벤트 기획
 - 럭키백은 구성을 모른채 먼저 구매하고, 배송받은 이후 구성품을 알 수 있는 상품
 - 럭키백 이벤트 소식을 들은 고객만족팀이 강하게 반대
- ➔ **럭키백에 포함된 생선의 확률을 알려주는 방향으로 이벤트 수정**



럭키백의 확률



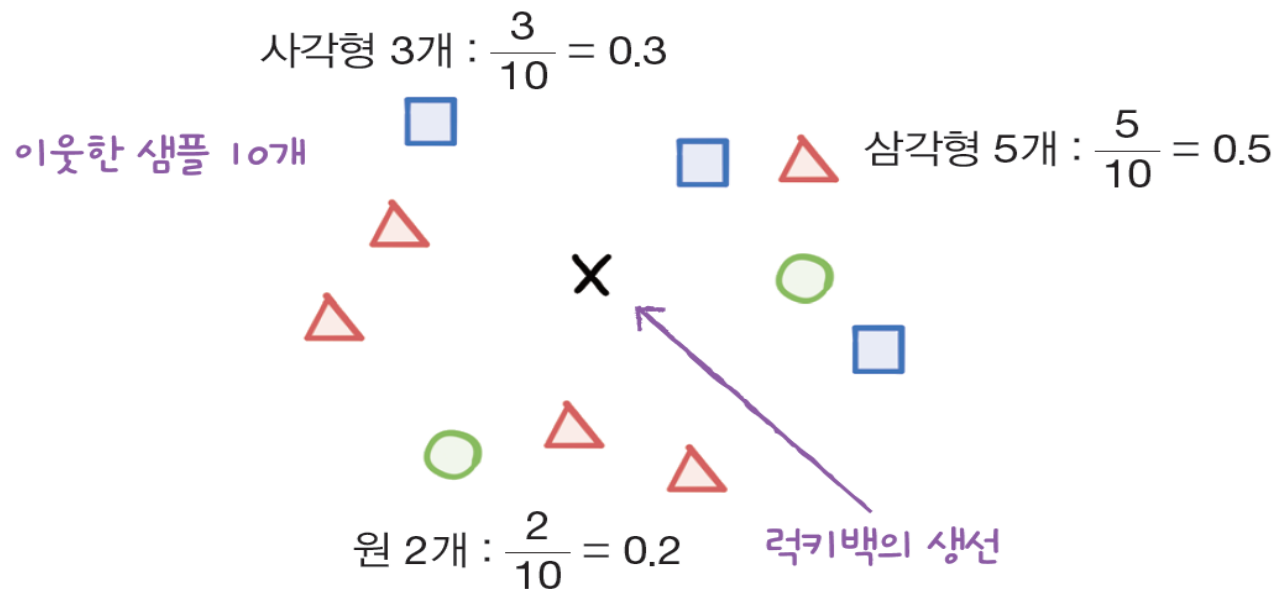
- 럭키백에 들어갈 수 있는 **생선은 7개**
- 럭키백에 들어간 생선의 크기, 무게 등이 주어졌을 때 7개 생선에 대한 확률을 출력
- 길이, 높이, 두께 외에도 대각선 길이와 무게를 사용하여 품종을 예측



럭키백의 확률



- “k-최근접 이웃은 주변 이웃을 찾아주니까 이웃의 클래스 비율을 확률이라고 출력하면?”
 - 샘플 주위에 가장 가까운 이웃 샘플 10개를 표시
 - 사각형이 3개, 삼각형이 5개, 원이 2개
 - 이웃한 샘플 X의 클래스를 확률로 삼는다면 샘플 X가 사각형일 확률은 30%, 삼각형일 확률은 50%, 원일 확률은 20%



럭키백의 확률



데이터 준비하기

- 판다스로 모델 훈련에 사용할 데이터를 만들기
- 판다스의 `read_csv()` 함수로 CSV 파일을 데이터프레임으로 변환

```
import pandas as pd
fish = pd.read_csv('https://bit.ly/fish_csv')
fish.head()
```

	Species	Weight	Length	Diagonal	Height	Width
0	Bream	242.0	25.4	30.0	11.5200	4.0200
1	Bream	290.0	26.3	31.2	12.4800	4.3056
2	Bream	340.0	26.5	31.1	12.3778	4.6961
3	Bream	363.0	29.0	33.5	12.7300	4.4555
4	Bream	430.0	29.0	34.0	12.4440	5.1340

럭키백의 확률



■ 데이터 준비하기

- 판다스의 `unique()` 함수를 사용하여, 어떤 종류의 생선이 있는지 Species 열에서 고유한 값을 추출

```
print(pd.unique(fish['Species'])) → ['Bream', 'Roach', 'Whitefish', 'Parkki', 'Perch', 'Pike', 'Smelt']
```

- 데이터프레임에서 Species 열을 타겟으로 만들고 나머지 5개 열은 입력 데이터로 사용

```
fish_input = fish[['Weight', 'Length', 'Diagonal', 'Height', 'Width']]
```

- 데이터프레임에서 여러 열을 선택하면 새로운 데이터프레임이 반환. 이를 `fish_input`에 저장

럭키백의 확률



■ 데이터 준비하기

- fish_input에 5개의 특성이 잘 저장되었는지 처음 5개 행을 출력

```
fish_input.head( )
```



	weight	length	diagonal	height	width
0	242.0	25.4	30.0	11.5200	4.0200
1	290.0	26.3	31.2	12.4800	4.3056
2	340.0	26.5	31.1	12.3778	4.6961
3	363.0	29.0	33.5	12.7300	4.4555
4	430.0	29.0	34.0	12.4440	5.1340

럭키백의 확률



▪ 데이터 준비하기

- 동일한 방식으로 타깃 데이터 준비

```
fish_target = fish['Species']
```

- 데이터를 훈련 세트와 테스트 세트로 나누기

```
from sklearn.model_selection import train_test_split
train_input, test_input, train_target, test_target = train_test_split(
    fish_input, fish_target, random_state=42)
```

- 사이킷런의 StandardScaler 클래스를 사용해 훈련 세트와 테스트 세트를 표준화 전처리

```
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
ss.fit(train_input)
train_scaled = ss.transform(train_input)
test_scaled = ss.transform(test_input)
```

럭키백의 확률



- k-최근접 이웃 분류기의 확률 예측
 - 사이킷런의 KNeighborsClassifier 클래스 객체를 만들고 훈련 세트로 모델을 훈련한 다음 훈련 세트와 테스트 세트의 점수를 확인
 - 최근접 이웃 개수인 k를 3으로 지정하여 사용

```
from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier(n_neighbors=3)
kn.fit(train_scaled, train_target)
print(kn.score(train_scaled, train_target))
print(kn.score(test_scaled, test_target))
```



0.8907563025210085
0.85

- 타깃 데이터를 만들 때 fish['Species']를 사용해 만들었기 때문에 훈련 세트와 테스트 세트의 타깃 데이터에도 7개의 생선 종류가 들어감
- 다중 분류(multiclass classification): 타깃 데이터에 2개 이상의 클래스가 포함

럭키백의 확률



- k-최근접 이웃 분류기의 확률 예측
 - 타깃값을 그대로 사이킷런 모델에 전달하면 순서가 자동으로 알파벳 순으로 매겨져 `pd.unique(fish['Species'])`로 출력했던 순서와 다르게 됨
 - `KNeighborsClassifier`에서 정렬된 타깃값은 `classes_` 속성에 저장됨

```
print(kn.classes_) → ['Bream' 'Parkki' 'Perch' 'Pike' 'Roach' 'Smelt' 'Whitefish']
```

- `predict()` 메서드 사용, 테스트 세트에 있는 처음 5개 샘플의 타깃값을 예측

```
print(kn.predict(test_scaled[:5])) → ['Perch' 'Smelt' 'Pike' 'Perch' 'Perch']
```

럭키백의 확률



- k-최근접 이웃 분류기의 확률 예측
 - 사이킷런의 분류 모델은 `predict_proba()` 메서드로 클래스별 확률값을 반환
 - 테스트 세트에 있는 처음 5개의 샘플에 대한 확률을 출력
 - 넘파이 `round()` 함수는 기본으로 소수점 첫째 자리에서 반올림을 하는데, `decimals` 매개변수로 유지할 소수점 아래 자릿수를 지정할 수 있음

```
import numpy as np
proba = kn.predict_proba(test_scaled[:5])
print(np.round(proba, decimals=4))
```

▲ 소수점 네 번째 자리까지 표기
다섯 번째 자리에서 반올림

→

[	0.	0.	1.	0.	0.	0.	0.]
[	0.	0.	0.	0.	0.	1.	0.]
[	0.	0.	0.	1.	0.	0.	0.]
[	0.	0.	0.6667	0.	0.3333	0.	0.]
[	0.	0.	0.6667	0.	0.3333	0.	0.]

럭키백의 확률



- k-최근접 이웃 분류기의 확률 예측
 - `predict_proba()` 메서드의 출력 순서는 앞서 보았던 `classes_` 속성과 같음
즉, 첫 번째 열이 'Bream'에 대한 확률, 두 번째 열이 'Parkki'에 대한 확률

첫 번째 클래스(Bream)에 대한 확률

↓

네 번째 샘플 → [0. 0. 0.6667 0. 0.3333 0. 0.]

↑

두 번째 클래스(Parkki)에 대한 확률

로지스틱 회귀



- 로지스틱 회귀(logistic regression)는 이름은 회귀이지만 **분류 모델**
- 이 알고리즘은 선형 회귀와 동일하게 선형 방정식을 학습

$$z = a \times (\text{Weight}) + b \times (\text{Length}) + c \times (\text{Diagonal}) + d \times (\text{Height}) + e \times (\text{Width}) + f$$

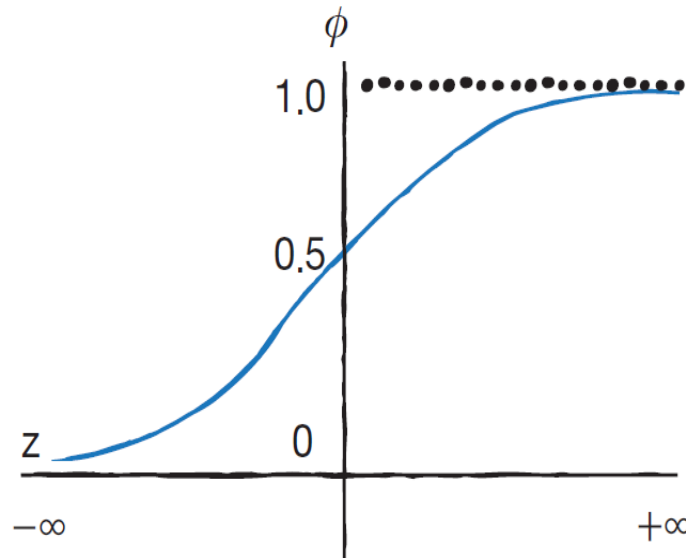
- a, b, c, d, e는 가중치 혹은 계수
- z는 어떤 값도 가능하지만 **확률이 되려면 0~1** (또는 0~100%) 사이 값이 되어야 함

로지스틱 회귀



- **시그모이드 함수(sigmoid function)** 또는 로지스틱 함수(logistic function)를 사용하여 z 가 아주 큰 음수일 때 0이 되고, z 가 아주 큰 양수일 때 1이 되도록 바꿀 수 있음

$$\phi = \frac{1}{1 + e^{-z}}$$



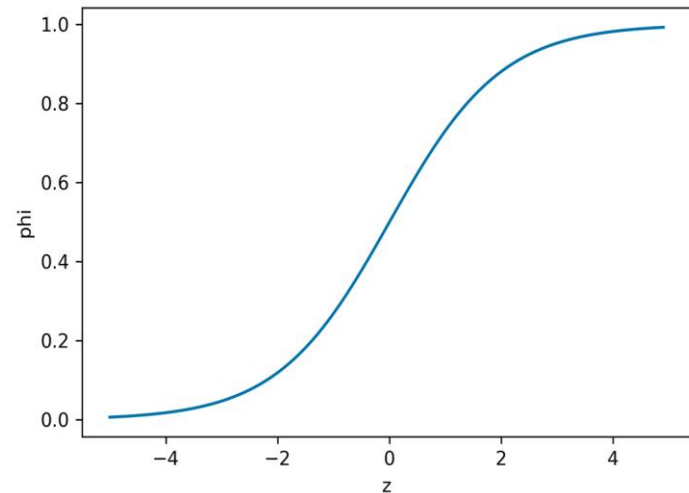
▲ 시그모이드 함수와 그래프

로지스틱 회귀



- 넘파이를 사용하여 시그모이드 그래프 작성
 - 먼저 -5와 5 사이에 0.1 간격으로 배열 z 를 만들고, 그다음 z 위치마다 시그모이드 함수를 계산
 - 지수 함수 계산은 `np.exp( )` 함수를 사용

```
import numpy as np
import matplotlib.pyplot as plt
z = np.arange(-5, 5, 0.1)
phi = 1 / (1 + np.exp(-z))
plt.plot(z, phi)
plt.xlabel('z')
plt.ylabel('phi')
plt.show()
```



- 이진 분류일 경우 시그모이드 함수의 출력값이 0.5보다 크면 양성 클래스, 0.5보다 작으면 음성 클래스로 판단
- 정확히 0.5일 때는 라이브러리마다 다를 수 있지만, 사이킷런은 0.5일 때 음성 클래스로 판단

로지스틱 회귀로 이진 분류 수행하기



- 불리언 인덱싱(Boolean indexing): 넘파이 배열은 True, False 값을 전달하여 행을 선택 가능
- 다음과 같이 'A'에서 'E'까지 5개의 원소로 이루어진 배열에서 'A'와 'C'만 골라내려면 첫 번째와 세 번째 원소만 True이고 나머지 원소는 모두 False인 배열을 전달

```
char_arr = np.array(['A', 'B', 'C', 'D', 'E'])  
print(char_arr[[True, False, True, False, False]])
```

→ ['A' 'C']

- 훈련 세트에서 도미(Bream)와 빙어(Smelt)의 행만 골라내기
 - 도미와 빙어에 대한 비교 결과를 비트 OR 연산자(|)를 사용해 결합

```
bream_smelt_indexes = (train_target == 'Bream') | (train_target == 'Smelt')  
train_bream_smelt = train_scaled[bream_smelt_indexes]  
target_bream_smelt = train_target[bream_smelt_indexes]
```

로지스틱 회귀로 이진 분류 수행하기



- 로지스틱 회귀 모델 훈련 - LogisticRegression 클래스는 선형 모델 `sklearn.linear_model` 패키지 안에 있음

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(train_bream_smelt, target_bream_smelt)
```

- 훈련한 모델을 사용해 `train_bream_smelt`에 있는 처음 5개 샘플을 예측

```
print(lr.predict(train_bream_smelt[:5]))
```

→ ['Bream' 'Smelt' 'Bream' 'Bream' 'Bream']

- `predict_proba()` 메서드로 `train_bream_smelt`에서 처음 5개 샘플의 예측 확률을 출력

```
print(lr.predict_proba(train_bream_smelt[:5]))
```

→

```
[[0.99760007 0.00239993]
 [0.02737325 0.97262675]
 [0.99486386 0.00513614]
 [0.98585047 0.01414953]
 [0.99767419 0.00232581]]
```

▲ 첫 번째 열이 음성 클래스(0)에 대한 확률, 두 번째 열이 양성 클래스(1)에 대한 확률

로지스틱 회귀로 이진 분류 수행하기



- 로지스틱 회귀가 학습한 계수 확인

```
print(lr.coef_, lr.intercept_) →
```

```
[[ -0.40451732 -0.57582787 -0.66248158 -1.01329614 -0.73123131]] [-2.16172774]
```

$$z = -0.405 \times (\text{Weight}) - 0.576 \times (\text{Length}) - 0.662 \times (\text{Diagonal}) - 1.013 \times (\text{Height}) - 0.731 \times (\text{Width}) - 2.161$$

- decision_function () 메서드로 train_bream_smelt의 처음 5개 샘플의 z값 출력

```
decisions = lr.decision_function(train_bream_smelt[:5])  
print(decisions) →
```

```
[-6.02991358  3.57043428 -5.26630496 -4.24382314 -6.06135688]
```

로지스틱 회귀로 다중 분류 수행하기



- LogisticRegression 클래스를 사용해 7개의 생선을 분류해 보면서 이진 분류와 차이점 학습
- LogisticRegression 클래스는 기본적으로 반복적인 알고리즘을 사용
 - max_iter 매개변수에서 반복 횟수를 지정하며 기본값은 100
 - 여기에 준비한 데이터셋을 사용해 모델을 훈련하면 반복 횟수가 부족하다는 경고가 발생
 - 충분하게 훈련시키기 위해 반복 횟수를 1,000으로 증가시킴



로지스틱 회귀로 다중 분류 수행하기

- LogisticRegression 클래스로 다중 분류 모델을 훈련하는 코드

```
lr = LogisticRegression(C=20, max_iter=1000)
lr.fit(train_scaled, train_target)
print(lr.score(train_scaled, train_target))
print(lr.score(test_scaled, test_target))
```

→ 0.9327731092436975
0.925

- 테스트 세트의 처음 5개 샘플에 대한 예측을 출력

```
print(lr.predict(test_scaled[:5]))
```

→ ['Perch' 'Smelt' 'Pike' 'Roach' 'Perch']

- 테스트 세트의 처음 5개 샘플에 대한 예측 확률을 출력
(소수점 네 번째 자리에서 반올림)

```
proba = lr.predict_proba(test_scaled[:5])
print(np.round(proba, decimals=3))
```

→

```
[[0.  0.014 0.842 0.  0.135 0.007 0.003]
 [0.  0.003 0.044 0.  0.007 0.946 0.  ]
 [0.  0.   0.034 0.934 0.015 0.016 0.  ]
 [0.011 0.034 0.305 0.006 0.567 0.   0.076]
 [0.  0.   0.904 0.002 0.089 0.002 0.001]]
```

로지스틱 회귀로 확률 예측 (문제해결 과정)



- 문제
 - 럭키백에 담긴 생선이 어떤 생선인지 확률을 예측하고 예측의 근거가 되는 확률을 출력
- 해결
 - 가장 대표적인 분류 알고리즘 중 하나인 로지스틱 회귀를 사용
 - 로지스틱 회귀는 회귀 모델이 아닌 분류 모델로 선형 회귀처럼 선형 방정식을 사용
 - 하지만 선형 회귀처럼 계산한 값을 그대로 출력하는 것이 아니라 로지스틱 회귀는 이 값을 0~1 사이로 압축(이 값은 마치 0~100% 사이의 확률)

로지스틱 회귀로 확률 예측 (문제해결 과정)



■ 해결

- 로지스틱 회귀는 이진 분류에서는 하나의 선형 방정식을 훈련. 이 방정식의 출력값을 시그모이드 함수에 통과시켜 0~1 사이의 값을 생성하고, 이 값이 양성 클래스에 대한 확률
(음성 클래스의 확률은 1에서 양성 클래스의 확률을 빼면 됨)
- 다중 분류일 경우에는 클래스 개수만큼 방정식을 훈련하고, 각 방정식의 출력값을 소프트맥스 함수를 통과시켜 전체 클래스에 대한 합이 항상 1이 되도록 만듦
 - 이 값은 각 클래스에 대한 확률로 이해할 수 있음



- 키워드로 끝나는 핵심 포인트
 - 로지스틱 회귀는 선형 방정식을 사용한 분류 알고리즘
 - 선형 회귀와 달리 시그모이드 함수나 소프트맥스 함수를 사용하여 클래스 확률을 출력
 - 다중 분류는 타깃 클래스가 2개 이상인 분류 문제
 - 로지스틱 회귀는 다중 분류를 위해 소프트맥스 함수를 사용하여 클래스를 예측
 - 시그모이드 함수는 선형 방정식의 출력을 0과 1 사이의 값으로 압축하며 이진 분류를 위해 사용
 - 소프트맥스 함수는 다중 분류에서 여러 선형 방정식의 출력 결과를 정규화하여 합이 1이 되도록 함

확인 문제



1. 로지스틱 회귀(Logistic Regression)의 주된 목적은?

- ① 데이터 정렬
- ② 데이터 분류
- ③ 데이터 압축
- ④ 데이터 시각화

2. 로지스틱 회귀에서 사용하는 활성화 함수는?

- ① ReLU
- ② Sigmoid
- ③ Softmax
- ④ Linear

확인 문제



3. 시그모이드 함수의 출력 범위는?

- ① $-1 \sim 1$
- ② $0 \sim 1$
- ③ $0 \sim 100$
- ④ $-\infty \sim \infty$

4. 시그모이드 함수의 출력이 0.85라면 의미는?

- ① 클래스 0일 확률 85%
- ② 클래스 1일 확률 85%
- ③ 정확도 85%
- ④ 오차율 85%

확인 문제



5. 다음 중 이진 분류(Binary Classification) 문제는?

- ① 붓꽃 3종 분류
- ② 손글씨 숫자 10종 분류
- ③ 스팸메일 여부 판별
- ④ 과일 종류 분류

6. 예측 확률이 0.72일 때 기본 기준으로 예측되는 클래스는?

- ① 0
- ② 1
- ③ 둘 다
- ④ 알 수 없음

확인 문제



7. 다음 중 로지스틱 회귀 모델을 생성하는 코드는?

- ① KNeighborsClassifier()
- ② DecisionTreeClassifier()
- ③ LogisticRegression()
- ④ LinearRegression()

8. lr.predict_proba(X) 코드의 의미는 ?

- ① 클래스 반환
- ② 특성 중요도 반환
- ③ 클래스별 확률 반환
- ④ 정확도 반환

확인 문제



9. lr.classes_ 속성의 의미는 ?

- ① 클래스 목록
- ② 정확도
- ③ 회귀계수
- ④ 특성 수

10. 시그모이드 함수의 입력값이 매우 작은 음수이면 출력값은?

- ① 0에 가까워짐
- ② 1에 가까워짐
- ③ 무한대
- ④ -1

확인 문제



11. `lr.score(test_scaled, test_target)` 코드의 의미는 ?

- ① 정확도 계산
- ② 확률 계산
- ③ 클래스 예측
- ④ 모델 저장

12. 로지스틱 회귀에서 확률 0.49와 0.51은 어떤 의미가 있는가?

- ① 둘 다 클래스 1
- ② 둘 다 클래스 0
- ③ 0.49는 클래스 0, 0.51은 클래스 1
- ④ 둘 다 분류 불가

Q & A

